FLEET UI · TESTE BLE COM O TURTLEBOT4 REAL · CIN-UFPE

Roteiro do teste no CIn

Do dia anterior até guardar os dados, na ordem. Marquem cada passo quando ele der certo; só sigam para o próximo com o anterior marcado. Os detalhes de instalação estão no manual.

Parem tudo se

- o robô sair do percurso, se aproximar de uma pessoa ou de um obstáculo: parada física primeiro (segurar ou levantar o robô), tela depois;
- a checagem do passo 11 tiver qualquer **FALHA**;
- a tela mostrar **Nav2 OFF** ou a posição do robô no mapa não bater com onde ele está.

O botão ■ **Parar** da tela cancela a navegação, mas o robô ainda anda cerca de 3 s (uns 80 cm) até parar.

ANTES DO DIA

No PC do laboratório

- [ ] **Docker instalado e funcionando sem sudo**

  Passo 1 do manual.

```
docker run hello-world
```

  Esperado: a mensagem “Hello from Docker!”.

- [ ] **Fleet UI baixado e montado**

  Passo 2 do manual (repositório ou o arquivo `.zip`). A montagem leva de 15 a 30 minutos: façam com antecedência.

```
cd ~/fleet-ui
make docker-build
```

  Esperado: termina com “Image fleet-ui:dev Built”.

- [ ] **Uma corrida completa na simulação**

  Confirma que tudo funciona nessa máquina antes de envolver o robô. Conectem o notebook na rede em que vão ficar **antes** de ligar: se o Wi-Fi mudar com a simulação ligada, ela para de responder; aí, Ctrl+C, `make docker-down` e `make cbsoft-up` de novo.

3

```
make cbsoft-up       # abrir http://localhost:5173, Conectar, ▶ Executar replay
make docker-down     # depois de Ctrl+C, para desligar
```

  > **Esperado:** "Sucesso", cerca de 96 m e 0,25 m/s no canto direito da tela.

- [ ] **Material separado**

  4 Robô carregado na base, os dois celulares carregados e pareados, PC Windows com o script de leitura BLE de sempre.

NO LABORATÓRIO Robô e rede

- [ ] **Robô ligado e iniciado**

  5 Esperem o robô terminar de iniciar, como vocês já fazem.

- [ ] **PC e robô na mesma rede**

  Conectem o PC no Wi-Fi **TurtleLab**, o do roteador portátil. Essa rede **não tem internet**: `git pull` e `make docker-build` têm de ser feitos antes, em outra rede.

  6 O robô é o `192.168.50.11`; o `192.168.50.1` é o roteador. Se o IP mudar, ele aparece na telinha do robô ou em `http://192.168.50.1`, na lista de aparelhos conectados.

```
ping 192.168.50.11
```

  > **Esperado:** respostas com tempo, sem "Destination Host Unreachable".

- [ ] **Nav2 de vocês desligado**

  O Nav2 do contêiner de vocês é Humble e não conversa com o do Fleet UI. Se ele estiver ligado:

7

```
docker stop ttb4
```

- [ ] **Relógio do robô igual ao do PC**

  O mesmo comando do guia de troubleshooting de vocês (pede a senha do robô). O painel do passo 11 também mostra esse comando pronto para copiar.

8

```
ssh ubuntu@192.168.50.11 "sudo date -s '@$(date +%s.%N)'"
```

LIGAR

Fleet UI no robô

- [ ] **Fleet UI ligado**

  Liga só a tela, ainda sem mexer no robô. Deixem esse terminal aberto. Em cerca de 1 minuto, abram `http://localhost:5173`.

```
cd ~/fleet-ui
make robot-up ROBOT_IP=192.168.50.11
```

  Esperado: à esquerda, o painel **Robô real**, com o IP já preenchido.

- [ ] **Robô no START**

  Coloquem o robô no ponto de partida de sempre, virado para a mesma direção das corridas de vocês.

- [ ] **Conectar no painel Robô real**

Cliquem em ▶ **Conectar**. O painel confere se o IP responde, faz a checagem pré-voo (só escuta o robô, não publica nada) e, se nada der FALHA, liga no PC o mapa, o AMCL e o Nav2.

> **Esperado:** em cerca de 30 s, **✓ Robô conectado** e "Checagem: 14 de 14 OK"; logo depois, **o robô gira um pouquinho (~10°) para um lado e volta**, no lugar, para mostrar que obedece ao PC.

Esse giro não anda para frente, mas deixem uns 30 cm livres em volta do robô. Para desligá-lo, liguem com `make robot-up ROBOT_IP=192.168.50.11 ROBOT_CONNECT_WIGGLE=false`.

Se aparecer **✗ Não conectou**, a lista mostra a linha com ✗. Depois de resolver, cliquem em ↻ **Reconectar**:

Linha	O que fazer
`rede`	O PC não alcança o IP: confiram se o PC está na TurtleLab (passo 6) e se o IP é o do robô, não o do roteador.
`middleware`	O robô precisa estar em CycloneDDS: `RMW_IMPLEMENTATION=rmw_cyclonedds_cpp` em `/etc/turtlebot4/setup.bash` e na base Create3, como no guia de vocês. Depois, `sudo systemctl restart turtlebot4.service` no robô.
`nós descobertos`	A rede passa, mas o ROS não acha o robô: domínio ou middleware. Se o robô não usa o domínio 0, desliguem com Ctrl+C e religuem com `make robot-up ROBOT_DOMAIN=<n> ROBOT_IP=192.168.50.11`.
`relógio`	Repetir o passo 8.
`TF odom → base_link`	No robô: `sudo systemctl restart turtlebot4.service` (erro 1 do guia de vocês).
`taxa /scan ou /odom`	Laser ou base sem publicar: reiniciar o serviço do robô, como acima.
`Nav2 já rodando (aviso)`	Repetir o passo 7.

Plano B, se o painel não conectar por outro motivo: Ctrl+C no terminal do passo 9 e o jeito antigo, pelo terminal: `make robot-check` e, com tudo OK, `make robot-up-terminal`; depois, na tela, **Conectar robô → Custom (simulação local) → Conectar**.

11

- [ ] **Tela confere com o robô**

> **Esperado:** no alto da tela, **Nav2 ON** e posição perto de x = 1,02 e y = 16,62 (o START).

Se o robô estiver em outro lugar, informem a posição real no campo **Pose inicial** (ela é enviada ao AMCL no começo da execução, como o "2D Pose Estimate" do RViz).

A **Pose inicial** não move o robô: só diz ao AMCL onde ele está. Ela só é certa se o robô estiver de fato naquele ponto, virado na direção informada. O botão ⊕ **Pose atual** copia onde o sistema já acha que o robô está; se essa posição estiver errada, ele copia o erro.

12

- [ ] **Executar liberado**

13 > **Esperado:** no topo, **● Conectado — robô real 192.168.50.11**, e o botão **▶ Executar** disponível. Não há mais o "Conectar robô → Custom" do topo: o painel já é a conexão.

PRIMEIRO MOVIMENTO

Teste curto

- [ ] **Andar 1 metro**

Com espaço livre e alguém perto do robô: em **Gravar**, cliquem no mapa cerca de 1 m à frente do robô e usem → **Navegar rota**, que anda sem gravar.

> **Esperado:** o robô anda até o ponto e para; o campo Nav volta para **idle**.

14 Depois, confiram duas vezes que dá para recomeçar sem religar o Fleet UI: levem o robô ao START, façam o passo 17 e parem com **■ Parar** depois de uns 5 m. Se nas duas vezes a posição no alto da tela foi para o START e o robô seguiu reto, usem o passo 21 sem religar. Se não, religuem entre as corridas (o plano B do passo 21).

A CORRIDA

Teste BLE

- [ ] **Celulares no lugar**

15 Um fixo no START, o outro em cima do robô, conectados, como nas corridas de vocês. O Fleet UI não mexe no Bluetooth: a conexão entre os celulares e a leitura do sinal continuam do jeito de sempre, fora dele. O painel **Robô real** da tela não tem relação com Bluetooth.

- [ ] **Leitura BLE começando no PC Windows**

Antes, confiram que o relógio do PC Windows está no horário automático e no fuso de Recife: a análise casa o sinal com a posição pelo horário.

16

```
.\script_celular.ps1
```

- [ ] **Executar o percurso**

  Na tela: **Reproduzir**, rota `percurso_cbsoft`, **Gravar bag** marcado, ▶ **Executar replay**. Na lista de tópicos, deixem `ble_rssi` **desmarcado**: é um recurso opcional que leria o Bluetooth pelo PC Linux e não foi testado com o robô real.

  Antes de executar, no campo **Pose inicial**: `1.02`, `16.62`, `0` (o START). Só com o robô de fato no START, virado como nas corridas de vocês.

  > **Esperado:** nos primeiros segundos, a posição no alto da tela vai para perto de x = 1,02 e y = 16,62; depois o robô segue o "L" (64 m e depois 31 m), a cerca de 0,25 m/s.

  Se a posição não for para o START, ou o robô virar para outro lado em vez de seguir reto, parem: parada física primeiro, ■ Parar depois. Informar a pose ao AMCL não garante que ele localizou o robô: ele ainda confere com o laser.

  Na lista de tópicos, `amcl_pose` precisa estar **marcado** (já vem assim): é a posição do robô no mapa, de onde sai o CSV do passo 20. Tópicos marcados que o robô real não tem, como `pose` e `ground_truth_pose_clean`, só gravam vazio.

- [ ] **Parar na desconexão**

  Quando o script do Windows registrar a desconexão, parem a corrida com ■ **Parar** na tela e o script com Ctrl+C, como vocês faziam com os dois terminais.

- [ ] **Log do celular copiado**

  Copiem o log do Windows para `fleet_ws/collections/cbsoft/nrf/` no PC do Fleet UI.

DEPOIS DE CADA CORRIDA

Análise e nova corrida

- [ ] **Gráfico de RSSI gerado**

```
make cbsoft-csv
make cbsoft-analise NAV=collections/cbsoft/<navegacao>.csv NRF="collections/cbsoft/nrf/<log>.txt"
```

> **Esperado:** um PNG ao lado do CSV, com o RSSI ao longo da posição.

- [ ] **Robô de volta ao START para a próxima**

Esperem o campo Nav voltar para **idle**, com o robô parado. Levem o robô ao START com as mãos, virado na mesma direção de sempre; uma marca de fita no chão ajuda a repetir o lugar. Deixem a **Pose inicial** no START (passo 17) e voltem ao passo 15, sem religar nada.

Plano B, se a verificação do passo 14 falhou ou a posição não for para o START: com o robô já no START, cliquem em ↻ **Reconectar** no painel Robô real (cerca de 30 s): o AMCL recomeça no START.

Anotem a cada corrida como o robô foi posto no START (marca no chão, direção). Colocar o robô à mão muda um pouco o ponto de partida de uma corrida para outra, e isso precisa constar nos resultados.

NO FIM

Encerrar e guardar

- [ ] **Tudo desligado e robô na base**

```
make docker-down
```

- [ ] **Dados copiados para outro lugar**

Copiem a pasta `fleet_ws/collections/` inteira (gravações, CSVs, logs e gráficos) para um pendrive ou para a nuvem antes de sair.

Até agora, a ligação com o robô foi testada com um robô falso, que publica laser, odometria e TF como o TurtleBot4: pelo painel Robô real, IP errado recusado, checagem OK, Nav2 ativo em cerca de 25 s, deslocamento de 1 m concluído, Reconectar sem deixar processos para trás, percurso com gravação de `/amcl_pose` e parada no meio funcionando. Esta é a primeira vez com o robô de verdade.

Fleet UI, repositório `fleet-ui-ufpe`. Dúvidas: falem com o Eduardo.